BioTrivial UAX.

Leila Bargach, Celia García Sánchez, Rocío Cepeda Muñoz y Valeria Encinas Díaz-Jiménez

BioTrivial es un juego educativo de preguntas y respuestas sobre asignaturas de Biomedicina. Está diseñado para dos jugadores y permite repasar cinco asignaturas. El tablero tiene 25 casillas de colores. En cada turno se lanza un dado virtual y se responde a una pregunta relacionada con el tema de la casilla. Si la respuesta es correcta, se puede volver a tirar, si no, se termina el turno y se pasa al siguiente jugador. Para ganar hay que llegar exactamente a la casilla final y responder correctamente una pregunta final aleatoria.
Para el desarrollo de este juego hemos utilizado funciones, listas, diccionarios, bucles, condiciones y la biblioteca random. También hemos añadido mensajes divertidos y un sistema para evitar que las preguntas se repitan continuamente.

1. Reglas del juego.

- Participan 2 jugadores, que deben introducir sus nombres al comenzar.
- Ambos empiezan en la posición 0 (inicio) y en cada turno se lanza un dado virtual del 1 al 6.
- Cada una de las 25 casillas tiene un color asociado a una categoría. Al caer en una casilla se responde una pregunta de esa materia.
- Si se acierta, el mismo jugador vuelve a lanzar el dado, pero si se falla el turno pasa al otro jugador. El jugador se queda en la casilla alcanzada.
- Si el lanzamiento supera la casilla 26 (final), el jugador no avanza y pierde su turno. Es obligatorio llegar con el número exacto.
- Al llegar a la casilla 26 (final) se hace una pregunta final de una categoría elegida al azar. Solo se gana si se responde correctamente. Si se falla esta pregunta, el jugador permanece en la casilla y en su siguiente turno intenta otra pregunta final sin tirar el dado.

2. Categorías.

Color.	Asignatura.
Azul	Genética
Verde	Fisiopatología
Rojo	Microbiología
Amarillo	Biología celular y molecular
Morado	Farmacología

El banco de preguntas contiene 50 preguntas, 10 por categoría. Las preguntas se seleccionan aleatoriamente y no se repiten dentro de una categoría hasta que se hayan utilizado todas las disponibles.

3. Organización del código.

Bloque.	¿Para qué sirve?
Importaciones	• Random genera resultados aleatorios. • Unicodedata permite normalizar acentos. • IPython.display muestra la imagen.
bienvenida()	Presenta el título y las instrucciones iniciales.
mostrar_reglas()	Presenta las reglas principales antes de comenzar la partida.
mostrar_tablero()	Muestra la imagen del tablero guardado como IMG_2875.jpeg.
mostrar_fichas()	Representa gráficamente las posiciones de los dos jugadores.
Tablero, categorías y fin	Definen el orden de colores, las materias y la posición final 26.
registrar_jugadores()	Pide los nombres de los dos jugadores.
Preguntas	Almacena en un diccionario las preguntas y sus respuestas.
mensajes_correctos/incorrectos	Ofrecen comentarios divertidos al azar.
normalizar_respuesta()	Ignora diferencias de mayúsculas, espacios extremos y tildes.
preparar_preguntas()	Crea y mezcla la lista de preguntas disponibles para evitar repeticiones.
hacer_pregunta()	Muestra la pregunta, recibe la respuesta, comprueba el resultado y devuelve True o False.
lanzar_dado y mover_jugador()	Simulan el dado y calculan el avance sin superar la casilla final.
jugar()	Coordina el turno, la pregunta de la casilla y el desafío final.
jugar() al final	Inicia la partida cuando se ejecuta la última celda.

4. Conceptos aplicados.

- Variables: guardan datos como FIN = 26 o el turno actual.
- Listas: almacenan colores, jugadores, posiciones y mensajes.
- Diccionarios: relacionan colores con asignaturas y guardan preguntas con respuestas.
- Funciones (def): dividen el programa en tareas reutilizables.
- Condicionales (if/else): determinan si una respuesta es correcta, si cambia el turno o si alguien gana.
- Bucle while True: mantiene la partida activa hasta que se cumple la condición de victoria; break la termina y continue inicia la siguiente vuelta.
- random.randint(1,6): simula el dado; random.choice(): selecciona una categoría o un comentario al azar.
- return True/False: comunica a la función principal si el jugador ha acertado.

Visualización y movimiento de fichas sobre el tablero.

Hemos añadido una representación visual de los jugadores mediante dos fichas que cambian de posición sobre el tablero a medida que avanza la partida. Para poder hacerlo utilizamos dos bibliotecas de Python: Matplotlib y Pillow (PIL).

Biblioteca Matplotlib: permite crear gráficos y representaciones visuales. La utilizamos para dibujar las fichas de los jugadores directamente sobre la imagen del tablero. Instrucciones utilizadas:

- import matplotlib.pyplot as plt: importa las herramientas de representación gráfica.
- plt.subplots (figsize=(12, 10)): crea la figura y el espacio donde se mostrará el tablero.
- ax.imshow(imagen): coloca la imagen original del tablero como fondo.
- ax.scatter(x, y, …): dibuja las fichas en las coordenadas correspondientes a cada casilla.
- ax.text(x, y, …): escribe el número de cada jugador dentro de su ficha.
- ax.axis("off "): oculta los ejes del gráfico para que solo se vea el tablero.
- plt.show(): muestra el resultado en Jupyter.

Biblioteca Pillow (PIL): permite abrir y manipular imágenes. Utilizamos la instrucción from PIL import Image as PILImage para importar sus funciones y PILImage.open (« IMG_2875.jpeg ») para abrir la imagen original del tablero. Además, las propiedades imagen.width e imagen.height permiten conocer las dimensiones de la imagen, necesarias para calcular la posición de las fichas.

Sistema de coordenadas: para que Python sepa dónde debe aparecer cada ficha, creamos una lista (coordenadas). Contiene 27 posiciones: la casilla inicio (posición 0), las 25 casillas del recorrido (posiciones 1 a 25) y la casilla final (posición 26). Utilizamos coordenadas proporcionales que después se multiplican por las dimensiones reales de la imagen:

- x, y = coordenadas[posiciones[i]]
- x = x * imagen.width
- y = y * imagen.height

Así podemos situar las fichas sobre las casillas del tablero. Las coordenadas se ajustaron teniendo en cuenta la distribución de las casillas en nuestra imagen original.

5. Preguntas.

¿Por qué utilizamos funciones?	Para dividir el código en partes comprensibles y reutilizables.
¿Por qué random?	Para simular un dado y elegir preguntas, categorías y mensajes aleatorios.
¿Qué hace while True?	Repite los turnos hasta que alguien supera la pregunta final y break termina el bucle.
¿Cómo se comprueba una respuesta?	Se normaliza el texto y se compara con la respuesta guardada; la función devuelve True o False.
¿Por qué un diccionario para las preguntas?	Porque permite acceder a cada conjunto de preguntas mediante el nombre de su categoría.
¿Cómo evitáis preguntas repetidas?	Se mezclan copias de las listas y se extrae cada pregunta; al agotarse una categoría se vuelve a cargar.

6. Distribución del trabajo entre los integrantes del grupo.

- Programación inicial: Rocío.
  - Desarrollo e integración de las funciones principales del programa.
  - Organización del código en Jupyter.
- Integración del juego: Leila.
  - Implementación de la lógica de turnos, movimiento de jugadores y condiciones de victoria.
  - Incorporación de las fichas de los jugadores. Comprobación de las coordenadas y de la visualización del movimiento.
  - Integración de las diferentes funcionalidades y realización de pruebas de funcionamiento.
- Elaboración de las preguntas: Celia.
  - Selección de las cinco categorías temáticas.
  - Elaboración y revisión de las preguntas y respuestas.
  - Organización de las preguntas en diccionarios. Comprobación del funcionamiento en caso de respuesta correcta e incorrecta.
- Diseño del tablero y representación gráfica: Valeria.
  - Diseño y organización visual del tablero.
  - Distribución de las casillas y sus colores.

Explicación detallada del código.

Celda 1: identificación del proyecto. **# BIOTRIVIAL UAX**
- Hemos añadido un comentario para identificar el nombre de nuestro proyecto.
- El símbolo '#', permite escribir comentarios en Python. Los comentarios no se ejecutan y sirven para organizar el programa y facilitar su comprensión.

Celda 2: función bienvenida(). **def bienvenida():**
- Creamos una función que muestra el título del juego, un mensaje de bienvenida y una frase de motivación para los participantes.
- Utilizamos 'def' para definir la función y varias instrucciones 'print()' para mostrar mensajes en pantalla.
- ¿Por qué utilizamos una función? Porque permite agrupar varias instrucciones bajo un mismo nombre. De esta manera, podemos mostrar toda la bienvenida escribiendo simplemente 'bienvenida()'.

Celda 3: función mostrar_reglas(). **def mostrar_reglas()**
- Añadimos una función que presenta las reglas principales del juego. Estas reglas explican cómo se participa, cómo se avanza por el tablero, qué ocurre al responder correctamente o incorrectamente y qué se necesita para ganar.
- Utilizamos una función con varias instrucciones 'print()' que muestran las reglas numeradas.

Celda 4: importación de las bibliotecas random y unicodedata. **import random import unicodedata**
- Importamos dos módulos de Python que utilizaremos posteriormente.
- 'random': permite generar números y realizar selecciones aleatorias. Lo utilizamos para simular el dado, elegir preguntas, mezclar su orden y seleccionar mensajes.
- 'unicodedata': permite trabajar con caracteres Unicode. En nuestro juego lo utilizamos para normalizar respuestas y evitar errores provocados por las tildes.

Celda 5: función mostrar_tablero().

```
from IPython.display import Image, display
def mostrar_tablero():
    print("\n ESTE ES EL TABLERO")
    display(Image(filename="IMG_2875.jpeg"))
```

- Creamos una función para mostrar la imagen original de nuestro tablero.
- Importamos 'Image' y 'display' desde 'IPython.display', que permite visualizar imágenes dentro de Jupyter. La instrucción 'Image(filename="IMG_2875.jpeg")' carga el archivo y 'display()' lo muestra en pantalla.
- El archivo 'IMG_2875.jpeg' debe encontrarse en la ubicación desde la que el programa pueda acceder a él.

Celda 6: representación gráfica y movimiento de las fichas.
- Importación de Matplotlib y Pillow. **import matplotlib.pyplot as plt from PIL import Image as PILImage**
  - Importamos dos bibliotecas que permiten trabajar con imágenes y representaciones gráficas.

    - Matplotlib permite dibujar elementos gráficos, mientras que Pillow permite abrir imágenes y consultar sus dimensiones.
    - Porque queríamos que los jugadores pudieran observar sus fichas directamente sobre el tablero y ver cómo cambiaban de posición durante la partida.
- Lista de coordenadas. **coordenadas = [(0.095, 0.13), (0.245, 0.13), (0.395, 0.13)]** (este fragmento representa únicamente las primeras posiciones de la lista).
    - Creamos una lista que almacena las coordenadas de las 27 posiciones del tablero: INICIO, las 25 casillas del recorrido y FINAL.
    - Cada elemento de la lista es una tupla formada por dos números (los valores son proporcionales al tamaño de la imagen):
        - 'x': posición horizontal.
        - 'y': posición vertical.
    - Para que el programa sepa dónde colocar cada ficha dependiendo de la posición del jugador.
- Función mostrar_fichas(). **'def mostrar_fichas(posiciones, jugadores):'**
    - Creamos una función que muestra el tablero con las dos fichas en sus posiciones correspondientes.
    - Primero abrimos la imagen original (**imagen = PILImage.open("IMG_2875.jpeg")**).
    - Después creamos la figura y mostramos el tablero (**fig, ax = plt.subplots(figsize=(12, 10)) ax.imshow(imagen)**), la variable 'fig' representa la figura completa y 'ax' el espacio sobre el que dibujamos.
    - Utilizamos un bucle (**for i in range(2)**). Este bucle permite realizar las mismas operaciones para los dos jugadores.
    - Para calcular la posición de cada ficha, utilizamos: **x, y = coordenadas[posiciones[i]] x = x * imagen.width y = y * imagen.height**. Multiplicamos las coordenadas proporcionales por las dimensiones de la imagen para convertirlas en posiciones reales expresadas en píxeles. Después dibujamos las fichas mediante 'ax.scatter()', que permite representar marcadores gráficos, y añadimos sus números utilizando 'ax.text()'. El jugador 1 tiene una ficha azul y el jugador 2 una ficha negra. Además, desplazamos ligeramente sus posiciones horizontales para que ambas fichas sean visibles cuando coinciden en una misma casilla.
    - Finalmente, utilizamos: **ax.axis("off") plt.show()**. Estas instrucciones ocultan los ejes y muestran el tablero con las fichas.

Las fichas no tienen una animación continua, sino que se actualizan después de cada movimiento.

Celda 7: definición del tablero, las categorías y la meta.

- Definimos los colores de las 25 casillas, las cinco categorías y la posición de la meta.
- Utilizamos una lista llamada 'tablero' que contiene los colores en el orden del recorrido.
- También creamos un diccionario llamado 'categorias', que relaciona cada color con una asignatura:
    - Azul: Genética.
    - Verde: Fisiopatología.
    - Rojo: Microbiología.
    - Amarillo: Biología celular y molecular.
    - Morado: Farmacología.
- Por último, definimos 'FIN = 26'.
- Para identificar qué categoría corresponde a la casilla en la que cae un jugador y determinar cuándo llega a la meta.
- Utilizamos un diccionario porque permite asociar cada color con su asignatura y recuperar esa información fácilmente.

Celda 8: función registrar_jugadores(). **def registrar_jugadores():**

- Creamos una función que solicita los nombres de los dos participantes.
- Utilizamos ‘input()’ para introducir los nombres y ‘.strip()’ para eliminar espacios innecesarios al principio y al final. También incorporamos condiciones ‘if’ para asignar los nombres «Jugador 1» y «Jugador 2» cuando el usuario no introduce ninguno. Finalmente, utilizamos ‘return jugador1, jugador2’ para devolver los nombres.

Celda 9: banco de preguntas.

- Creamos un banco de 50 preguntas relacionadas con las cinco categorías, diez preguntas por categoría.
- Utilizamos un diccionario principal donde cada categoría contiene una lista de preguntas. Cada pregunta se representa mediante otro diccionario con dos claves:
  - {
  - "pregunta": "¿Qué molécula contiene la información genética?",
  - "respuesta": "ADN"
  - }
- Facilita la selección de preguntas según la categoría y permite añadir nuevas preguntas sin modificar la lógica principal del juego.

Celda 10: mensajes para respuestas correctas e incorrectas.

- Creamos dos listas de mensajes divertidos: una para felicitar a los jugadores cuando aciertan y otra para reaccionar cuando fallan.
- Almacenamos diferentes frases en dos listas. Después, el programa utiliza ‘random.choice()’ para seleccionar un mensaje aleatorio de la lista correspondiente.

Celda 11: tratamiento y selección de preguntas.

- Función **normalizar_respuesta()**.
  - Creamos una función que prepara las respuestas para compararlas correctamente.
  - Utilizamos ‘.strip()’ para eliminar espacios exteriores y ‘.lower()’ para convertir las letras en minúsculas. Después aplicamos ‘unicodedata.normalize("NFD", texto)’ para separar los caracteres de sus marcas diacríticas (para quitar los acentos de las respuestas antes de compararlas). Mediante ‘unicodedata.category()’ y una comprensión de cadena, eliminamos las marcas correspondientes a las tildes.
  - Para evitar que una respuesta correcta se considere incorrecta únicamente por diferencias de mayúsculas, espacios exteriores o acentos.
  - Esta función no reconoce automáticamente sinónimos o respuestas científicamente equivalentes que estén escritas con palabras diferentes.
- Función **preparar_preguntas()**.
  - Creamos una función que prepara las preguntas antes de comenzar la partida.
  - Recorremos las categorías mediante un bucle ‘for’. Utilizamos ‘.copy()’ para crear copias de las listas originales y ‘random.shuffle()’ para mezclar aleatoriamente las preguntas de cada categoría.
  - Para que las preguntas aparezcan en un orden diferente y no se repitan dentro de una categoría hasta agotar las disponibles.
- Función **hacer_pregunta()**.

- Creamos una función que selecciona una pregunta, solicita la respuesta y comprueba si es correcta.
- Primero comprobamos si quedan preguntas disponibles en la categoría. Si se han agotado, volvemos a copiar y mezclar la lista original. Después utilizamos '.pop()' para extraer una pregunta de la lista y 'input()' para solicitar la respuesta del jugador. Comparamos la respuesta introducida con la respuesta correcta utilizando 'normalizar_respuesta()'. Si coinciden, mostramos un mensaje positivo y devolvemos 'True'. Si no coinciden, mostramos un mensaje divertido, indicamos la respuesta correcta y devolvemos 'False'.

Celda 12: lanzamiento del dado y movimiento.
- Función lanzar_dado(). **dado = random.randint(1, 6)**
  - Programamos un dado virtual de seis caras.
  - Utilizamos 'random.randint(1, 6)' para generar un número entero aleatorio entre 1 y 6, incluidos ambos extremos.
- Función **mover_jugador()**.
  - Creamos una función que calcula la nueva posición del jugador.
  - Sumamos el resultado del dado a su posición actual. Después comprobamos si la nueva posición supera 'FIN'. Si la supera, la función devuelve la posición anterior. En caso contrario, devuelve la nueva posición.

Celda 13: función principal jugar(). **def jugar():**

Al comenzar, ejecutamos las funciones 'bienvenida()', 'mostrar_reglas()' y 'mostrar_tablero()'. Después registramos a los participantes y guardamos sus nombres en una lista llamada 'jugadores'. Creamos otra lista, 'posiciones = [0, 0]', para indicar que ambos jugadores comienzan en INICIO. También preparamos las preguntas y establecemos 'turno = 0' para que comience el primer jugador. Y mostramos las dos fichas en la casilla inicial.

Bucle principal. **while True:**
- Utilizamos un bucle que mantiene la partida en funcionamiento hasta que uno de los jugadores gana. Dentro de este bucle se ejecutan los turnos, los lanzamientos del dado, las preguntas y las comprobaciones de victoria.

Control de los turnos. **turno = (turno + 1) % 2**
- Esta instrucción permite alternar entre los dos jugadores.
- El operador '%' calcula el resto de una división. Como solo existen dos jugadores, los valores del turno alternan entre 0 y 1. Si el jugador responde correctamente, el turno no cambia y puede volver a lanzar. Si falla, pasa al otro jugador.

Movimiento y actualización de las fichas.
- Durante el turno, el programa espera a que el jugador pulse ENTER para lanzar el dado.
- Después llama a 'lanzar_dado()' y 'mover_jugador()' para calcular la nueva posición.
- Si el movimiento es válido, actualizamos la lista 'posiciones' y ejecutamos 'mostrar_fichas(posiciones, jugadores)'.

Selección de la categoría. **color = tablero[nueva_posicion - 1] categoria = categorias[color]**
- Utilizamos la posición del jugador para consultar el color de su casilla y obtener la categoría correspondiente.

- Restamos 1 porque las listas de Python empiezan en el índice 0, mientras que las casillas del recorrido comienzan en la posición 1.
- Después llamamos a ‘hacer_pregunta()’ para evaluar al jugador.

Pregunta final y victoria.
- Cuando un jugador llega exactamente a la posición 26, el programa actualiza su ficha en FINAL.
- Sin embargo, llegar a la meta no garantiza la victoria.

categoria_final = random.choice(list(categorias.values()))
- Esta instrucción selecciona aleatoriamente una categoría entre las cinco asignaturas.
- El jugador debe responder una pregunta de esa categoría.
- Si responde correctamente, el programa muestra el mensaje de victoria y utiliza ‘break’ para terminar el bucle. Si falla, permanece en FINAL y el turno pasa al otro jugador mediante ‘continue’. Cuando vuelva a ser su turno, podrá intentar otra pregunta final sin lanzar el dado.

Finalización de la partida.
- Cuando un jugador supera la pregunta final, se ejecuta ‘break’, que permite salir del bucle principal.
- Después se muestra el mensaje de finalización y agradecimiento por participar.

Celda 14: ejecución del programa. **jugar()**
- Permite iniciar el juego completo.